Q1. State the Four Values of the Agile Manifesto. List Any Six of the Twelve Agile Principles.

Four Values of the Agile Manifesto

Definition: The Agile Manifesto is a set of values and principles that guide software development. It focuses on flexibility, teamwork, customer satisfaction, and delivering working software.

The four values are:

1. Individuals and interactions over processes and tools: Team communication and collaboration are more important than depending only on tools and processes.
2. Working software over comprehensive documentation: Delivering working software is more important than preparing excessive documentation.
3. Customer collaboration over contract negotiation: Developers should work closely with customers to understand and satisfy their needs.
4. Responding to change over following a plan: Teams should adapt to changing requirements instead of strictly following the original plan.

Note: The items on the right still have value; the Agile Manifesto gives greater importance to the items on the left.

Any Six Agile Principles

1. Customer satisfaction: Deliver valuable software early and continuously.
2. Welcome changes: Accept changing requirements, even late in development.
3. Frequent delivery: Deliver working software frequently.
4. Team collaboration: Business people and developers should work together daily.
5. Motivated individuals: Give team members support, trust, and a suitable environment.
6. Continuous improvement: Teams should regularly reflect on their work and improve their process.

Q2. Explain the "Three Ways" of DevOps from The DevOps Handbook. Give One Engineering Practice for Each Way. How Does Lean Thinking Underpin Them?

Definition: The Three Ways of DevOps are principles that help organizations deliver software faster, improve quality, and create a culture of continuous learning. They are Flow, Feedback, and Continuous Learning and Experimentation.

1. The First Way — Flow

The First Way focuses on the smooth and fast movement of work from development to operations and delivery.

- Goal: Reduce delays and bottlenecks in software delivery.
- Practice: Continuous Integration (CI), where developers frequently merge code and automatically build and test it.
- Example: Every code push triggers an automated build and test process.

2. The Second Way — Feedback

The Second Way focuses on creating fast feedback between every stage of development and operations.

- Goal: Detect and correct problems early.
- Practice: Monitoring and alerting using tools such as Prometheus and Grafana.
- Example: If server CPU usage becomes too high, an alert informs the team so it can investigate quickly.

3. The Third Way — Continuous Learning and Experimentation

The Third Way promotes continuous improvement, learning from failures, and safely trying new ideas.

- Goal: Improve the system and prevent repeated mistakes.
- Practice: Blameless postmortems after incidents.
- Example: After a deployment failure, the team identifies the root cause and adds automated tests to prevent the same issue from happening again.

Role of Lean Thinking

Lean thinking aims to deliver value to customers while reducing waste.

1. Flow: Lean reduces waiting time, handoffs, and bottlenecks so work moves smoothly.
2. Waste elimination: It removes unnecessary tasks, repeated manual work, and delays.
3. Fast feedback: Early detection prevents defects from becoming more expensive to fix.
4. Continuous improvement: Teams use lessons from failures to improve their processes.

Example: Automating software testing reduces manual effort, provides quick feedback, and helps code move faster from development to production. This demonstrates how Lean thinking supports all Three Ways of DevOps.

Q3. A company announces it has "adopted DevOps" by buying Jenkins and hiring a separate DevOps team. Developers still hand code to Operations at the end of each quarter, and production incidents are blamed on whoever deployed last. Analyze why this adoption is failing. Identify at least four DevOps failure modes in the scenario and propose a cultural and team-responsibility fix for each.

Short Exam-Style Question

Q3. Analyze Why DevOps Adoption Is Failing. Identify Four Failure Modes and Propose Fixes. (5 Marks)

Definition: DevOps is a culture and set of practices that bring Development and Operations teams together to deliver software quickly and reliably. Buying tools alone does not mean an organization has adopted DevOps.

Failure Modes and Their Fixes

Failure mode	Explanation	Cultural and team-responsibility fix
1. Tool-focused adoption	The company buys Jenkins but does not change its working practices.	Focus on collaboration, automation, and shared goals, not just tools.
2. Separate DevOps team	Developers and Operations work separately, creating silos.	Create cross-functional teams where Development and Operations share responsibility for delivery.
3. Manual handoffs	Developers hand over code to Operations quarterly, causing delays.	Use CI/CD pipelines to automate building, testing, and deployment.

4. Blame culture	Employees are blamed for production incidents, discouraging honest reporting.	Introduce blameless postmortems to find root causes and prevent repeated failures.
5. Lack of shared ownership	Developers focus only on writing code, while Operations handles all production problems.	Make teams jointly responsible for application reliability, monitoring, and production support.

Explanation of the Fixes

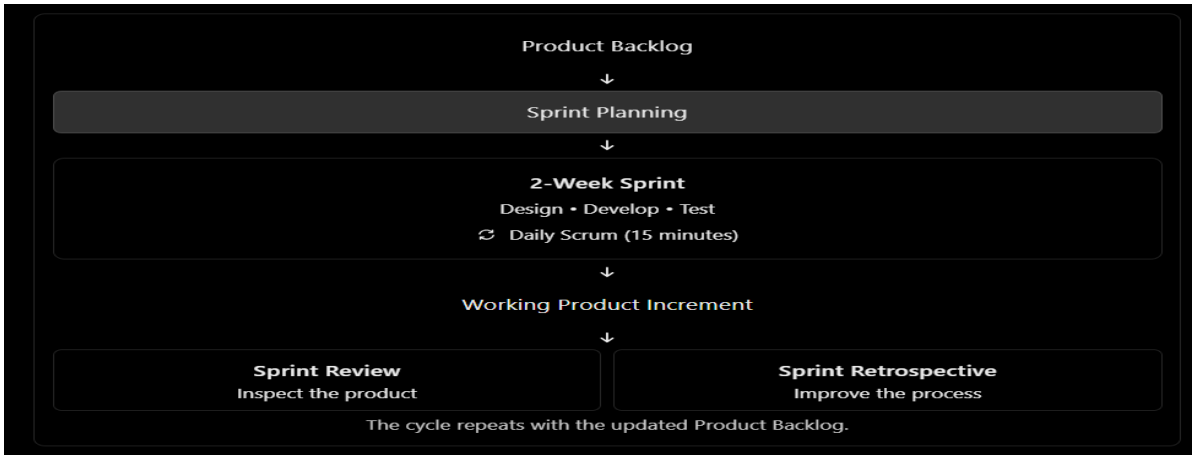
1. Collaboration: Developers, testers, and Operations should communicate regularly and work toward common objectives.
2. Automation: Jenkins should be used to build, test, and deploy code automatically rather than only as a purchased tool.
3. Shared responsibility: Teams should monitor applications and help resolve production issues together.
4. Continuous learning: After an incident, the team should analyze what went wrong and improve the system instead of blaming an individual.

Example: If a new release causes an application failure, the team investigates the root cause, fixes the pipeline or testing gap, and improves monitoring. Everyone learns from the incident.

Q4. Explain the Scrum Framework with a Neat Diagram of the Sprint Cycle. Describe the Three Roles, Three Artifacts, and Five Events, Including the Time-Box of Each Event for a 2-Week Sprint. (5 Marks)

Definition: Scrum is an Agile framework used to develop, deliver, and improve products through short, fixed-length iterations called Sprints. Each Sprint produces a usable product increment.

1. Scrum Sprint Cycle



2. Three Scrum Roles

1. Product Owner: Defines product goals, manages the Product Backlog, and prioritizes work.
2. Scrum Master: Helps the team follow Scrum, removes impediments, and improves team effectiveness.
3. Developers: Build, test, and deliver a usable product increment during each Sprint.

3. Three Scrum Artifacts

1. Product Backlog: An ordered list of all required product features and improvements.
2. Sprint Backlog: Selected backlog items and the plan for completing them in the current Sprint.
3. Increment: The usable product version completed during the Sprint that meets the Definition of Done.

4. Five Scrum Events and Time-Boxes

Event	Purpose	Time-box for a 2-week Sprint
Sprint	Complete the Sprint Goal	2 weeks
Sprint Planning	Select work and plan the Sprint	Up to 4 hours
Daily Scrum	Inspect progress and adapt the plan	15 minutes daily
Sprint Review	Inspect the increment and gather feedback	Up to 2 hours
Sprint Retrospective	Identify process improvements	Up to 1.5 hours

Q5. (a) Write Three User Stories for an Online Examination Portal with Acceptance Criteria. Check One Story Against INVEST. (b) Calculate Average Velocity, Number of Sprints, and Release Duration. Recalculate if Capacity Drops by 20%.

(a) Three User Stories and Acceptance Criteria

Definition: A user story describes a feature from the user's point of view and explains the value it provides.

User Story 1: Student Login

User story: As a student, I want to log in securely so that I can access my examinations.

Acceptance criteria:

- Given I have valid login details, When I enter them, Then I should access my student dashboard.
- Given I enter an incorrect password, When I submit the form, Then an error message should appear.

User Story 2: Take an Examination

User story: As a student, I want to start an examination so that I can answer questions online.

Acceptance criteria:

- Given an examination is available, When I click Start Exam, Then the examination questions should appear.
- Given the examination timer is running, When the time expires, Then the system should automatically submit my answers.

User Story 3: View Results

User story: As a student, I want to view my examination results so that I can check my performance.

Acceptance criteria:

- Given my result has been published, When I open the Results page, Then my marks should be displayed.
- Given my result has not been published, When I open the Results page, Then the system should show that the result is unavailable.

INVEST Check — Student Login Story

The INVEST criteria help ensure that a user story is well-written.

Letter	Meaning	Check
I	Independent	Login can be developed as a separate feature.
N	Negotiable	Details can be discussed with the team.
V	Valuable	It allows students to access the portal.
E	Estimable	The team can estimate the work required.
S	Small	The story can fit within a Sprint.
T	Testable	Acceptance criteria define testable outcomes.

The Student Login story satisfies all six INVEST criteria.

(b) Velocity and Release Calculation

Given:

- Sprint velocities: 28, 32, 30, and 34 story points
- Remaining Product Backlog = 186 story points
- Sprint duration = 2 weeks

Step 1: Calculate Average Velocity

$$\text{Average Velocity} = \frac{28 + 32 + 30 + 34}{4}$$

$$= \frac{124}{4} = \boxed{31 \text{ points}}$$

Step 2: Calculate Number of Sprints

$$\text{Sprints} = \frac{186}{31} = 6$$

Therefore, **6 Sprints** are needed.

Step 3: Calculate Release Duration

$$\text{Duration} = 6 \times 2 = \boxed{12 \text{ weeks}}$$

If One Team Member Leaves (20% Capacity Reduction)

Assuming velocity reduces proportionally with team capacity:

Step 1: New Velocity

$$31 \times (1 - 0.20) = \boxed{24.8 \text{ points}}$$

Step 2: New Number of Sprints

$$\frac{186}{24.8} = 7.5 \text{ Sprints}$$

Since a partial Sprint cannot be counted as a complete Sprint for planning, round up to **8 Sprints**.

Step 3: New Release Duration

$$8 \times 2 = \boxed{16 \text{ weeks}}$$

Measurement	Original team	Reduced team
Average velocity	31 points	24.8 points
Sprints required	6	8
Release duration	12 weeks	16 weeks

Q6. Compare Scrum and Kanban on Cadence, Roles, WIP Limits, Change Policy, and Key Metrics. Recommend a Framework for a Production-Support Team and Explain Retrospectives. (5 Marks)

Definition: Scrum and Kanban are Agile approaches used to manage work. Scrum uses fixed-length Sprints, while Kanban focuses on continuous workflow and limiting work in progress.

1. Scrum vs. Kanban

Basis	Scrum	Kanban
Cadence	Fixed-length Sprints, usually 1–4 weeks	Continuous flow; no fixed Sprint required
Roles	Product Owner, Scrum Master, Developers	No mandatory special roles
WIP limits	Sprint scope and team capacity guide work; no universal per-stage WIP limit	Explicit WIP limits are set for workflow stages
Change policy	Sprint Goal is protected; scope can be clarified or renegotiated without endangering the goal	New work can enter when capacity is available, according to team policies
Key metrics	Velocity and Sprint Goal progress	Lead time, cycle time, and throughput

2. Best Framework for Production Support

Recommendation: Kanban

A production-support team receives unpredictable and urgent tickets every day. Kanban is more suitable because it supports continuous incoming work without requiring the team to wait for the next Sprint.

Reasons:

1. Flexible priorities: Urgent incidents can be handled immediately.
2. Continuous workflow: Tickets move from To Do to In Progress to Testing and Done.
3. WIP limits: Limits prevent team members from handling too many tickets simultaneously.
4. Faster delivery: Cycle time helps measure how quickly tickets are resolved.
5. Better visibility: A Kanban board shows pending, active, and completed work.

Example Kanban Board:

To Do	In Progress	Done
Incoming tickets	WIP limit: 3	Resolved tickets

Example workflow: tickets move from left to right as work progresses.

3. How Does the Team Run Retrospectives?

Even without Scrum Sprints, the team can hold a weekly or monthly retrospective to review its workflow.

1. Discuss what went well.
2. Identify delays, recurring incidents, and bottlenecks.
3. Review cycle time and ticket throughput.
4. Agree on one or two improvements.
5. Check progress on those improvements at the next retrospective.

Example: If tickets remain in Testing for too long, the team can improve automated testing or arrange earlier tester involvement.

Q7. List and State the Purpose of Git Commands. Explain the Three Areas of Git. (5 Marks)

Definition: Git is a distributed version control system used to track code changes, manage different versions, and support collaboration among developers.

1. Important Git Commands and Their Purposes

Git command	Purpose
git init	Creates a new Git repository.
git clone URL	Copies an existing remote repository to the local computer.
git status	Shows modified, staged, and untracked files.
git add file	Adds changes to the staging area. Use git add . to stage all changes in the current directory.
git commit -m "message"	Saves staged changes as a commit.
git log	Displays commit history.
git branch	Lists branches; git branch name creates a branch.
git switch branch	Switches to another branch.
git checkout branch	Switches branches; it can also restore files in older workflows.
git merge branch	Combines changes from another branch into the current branch.
git pull	Fetches remote changes and integrates them into the current branch.
git push	Uploads local commits to a remote repository.

git stash	Temporarily saves uncommitted changes for later use.

2. Three Areas of Git

1. Working Directory: The folder where developers create, edit, and delete files.

2. Staging Area: The area where developers select changes to include in the next commit using git add.

3. Repository: The Git database where committed changes and project history are stored.

Git Workflow Diagram



Q8. Explain the Difference Between git merge and git rebase with Commit-Graph Diagrams. What Are Fast-Forward and Three-Way Merges? State the Golden Rule of Rebasing. (5 Marks)

Definition: Git merge and Git rebase are used to integrate changes from one branch into another. Merge preserves branch history, while rebase moves commits onto a new base to create a more linear history.

1. Difference Between Merge and Rebase

Basis	Git Merge	Git Rebase
Purpose	Combines changes from branches	Reapplies commits onto another base
History	May preserve a branching history	Creates a more linear history
Commit IDs	Existing commits are preserved	Rebased commits get new IDs
Merge commit	May create a merge commit	Does not create a merge commit for the rebase itself
Best use	Integrating shared branches	Cleaning up private feature-branch history

2. Commit-Graph Diagram

Suppose main has commits A → B, while the feature branch has commits C → D.

Before integration:

A---B--------- main

\

C---D--- feature

After git merge feature (three-way merge):

A---B-------M--- main

\ /

C---D

- M is a merge commit that joins both histories.
- The original commits C and D remain unchanged.

After rebasing the feature branch onto main:

A---B---C'---D'--- feature

- C' and D' are new commits created by replaying the feature changes on top of B.
- The feature history becomes linear.

3. Types of Merge

Fast-Forward Merge: Occurs when the target branch has no new commits since the feature branch diverged. Git simply moves the target branch pointer forward.

Before:

A---B---C---D feature

^

main

After:

A---B---C---D main, feature

Three-Way Merge: Occurs when both branches have new commits. Git uses their common ancestor and the two branch tips to combine the changes, usually creating a merge commit.

C---D feature

/ \

A---B-------M main

4. Golden Rule of Rebasing

Never rebase commits that other people may already have based their work on or are using.

Why?

1. Rebase changes commit IDs because it creates new commits.
2. Other developers may still have the original commits.
3. This can create duplicate-looking commits and confusing histories.
4. It may require complicated synchronization or recovery.

Example: Rebasing a private feature branch before sharing it is generally safe. Rebasing a shared branch such as main can disrupt teammates who have already pulled it.

Q9. Write the Complete Sequence of Git Commands to Add a Login Feature, Rebase, Resolve a Conflict, Push the Branch, and Open a Pull Request. (5 Marks)

Definition: Git branching allows developers to work on features separately. Rebasing updates a feature branch with the latest changes from main, while a pull request (PR) allows others to review the code before merging.

Step 1: Update main and Create a Feature Branch

git switch main

git pull --ff-only origin main

git switch -c feature/login

Step 2: Make Two Commits

After implementing the login feature in the project files:

git add login.html app.py

git commit -m "Add login page"

git add app.py

git commit -m "Add login validation"

Step 3: Rebase onto the Latest main

git fetch origin

git rebase origin/main

If a conflict occurs in app.py, Git pauses the rebase and displays a conflict message.

Step 4: Understand and Resolve the Conflict

Git may mark the conflicting code like this:

<<<<<<< HEADmessage = "Welcome to the application"=======message = "Welcome back, user">>>>>>> <commit-id>

- <<<<<<< HEAD marks the version from the new base (main) during this rebase.
- ======= separates the two versions.
- >>>>>>> <commit-id> marks the feature commit being replayed.

Resolve the conflict by editing app.py and keeping the correct code. For example:

message = "Welcome back, user"

Remove all conflict markers and save the file.

Then continue the rebase:

git add app.py

git rebase --continue

If more conflicts appear, resolve each one and repeat these commands. To cancel the rebase if necessary:

git rebase --abort

Step 5: Push the Feature Branch

git push -u origin feature/login

If this branch was already pushed before the rebase, rebasing rewrites its history. After coordinating with teammates, use git push --force-with-lease rather than a plain force-push.

Step 6: Open a Pull Request

Create a PR on GitHub or GitLab with:

- Source branch: feature/login
- Target branch: main
- Title: Add login feature
- Description: Explain the login functionality and testing performed.

After code review and required checks pass, merge the PR into main.

Q10. Compare GitFlow, GitHub Flow, and Trunk-Based Development. Recommend a Strategy for (a) a SaaS Team Deploying 10 Times a Day and (b) a Banking Product with Quarterly, Versioned Releases. (5 Marks)

Definition: Git branching strategies define how developers create branches, integrate code, test changes, and prepare software releases.

1. Comparison of Git Branching Strategies

Basis	GitFlow	GitHub Flow	Trunk-Based Development
Branch types	main, develop, feature, release, hotfix	main and short-lived feature branches	Shared trunk (main) and very short-lived branches, or direct commits
Release handling	Dedicated release branches and versioned releases	Deploy from main after checks pass	Frequent integration and automated releases
Merge frequency	Less frequent; branches may live longer	Frequent PR merges	Very frequent, ideally daily or several times a day
CI fit	Supports CI, but long-lived branches may create integration problems	Excellent fit with automated tests and PR checks	Strong fit with continuous integration and automated testing
Best suited for	Products with planned, versioned releases	Web applications and SaaS products	Teams needing rapid integration and delivery

2. Recommended Strategy

(a) SaaS Team Deploying 10 Times a Day — Trunk-Based Development

Reasons:

1. Developers integrate small changes into main frequently.

2. Automated tests detect problems early.
3. Short-lived branches reduce merge conflicts.
4. Continuous deployment enables frequent releases.
5. Feature flags allow incomplete features to remain disabled in production.

Example: A SaaS team merges a small, tested change into main and deploys it automatically within minutes.

(b) Banking Product with Quarterly, Versioned Releases — GitFlow

Reasons:

1. Dedicated release branches support planned quarterly releases.
2. The team can stabilize and test a release without stopping new development.
3. Hotfix branches support urgent production fixes.
4. Versioned releases make it easier to track changes.
5. Additional security, compliance, and regression testing can be completed before release.

Example: A bank prepares version v2.0 on a release branch, completes security testing, and deploys it after approval.

Note: GitFlow is a reasonable fit when quarterly releases require formal stabilization. The bank must still maintain strong automated testing and security checks.

Q11. Design a Pull-Request and Code-Review Policy for the main Branch of a 15-Developer Team on GitHub or GitLab. Specify Protected-Branch Rules and a PR Template. Explain Which Failure Each Rule Prevents. (5 Marks)

Definition: A pull request (PR) is a request to merge changes from one branch into another. Code review allows team members to check code quality, security, correctness, and maintainability before merging.

1. Protected-Branch Rules for main

Rule	Policy	Failure prevented
Required reviewers	Require at least 2 approvals before merging.	Prevents unreviewed or incorrect code from entering main.
CODEOWNERS	Require approval from the designated code owner for affected files.	Prevents changes to sensitive modules without expert review.
Required status checks	Require successful CI build, automated tests, linting, and security checks.	Prevents broken, poorly formatted, or vulnerable code from being merged.
Linear history	Require linear history; use squash or rebase merging and disallow merge commits if needed.	Prevents unnecessarily complicated commit histories.
No force-push	Block force-pushes to main.	Prevents published commits from being overwritten or lost.

Signed commits	Require verified signed commits.	Helps verify commit authenticity and detect unsigned or unverified commits.
No direct pushes	Require all changes to enter through approved PRs.	Prevents bypassing code review and automated checks.
Resolve conversations	Require all review conversations to be resolved before merging.	Prevents important review feedback from being ignored.

2. Example CODEOWNERS File

Create a file named .github/CODEOWNERS:

Backend code owners

/app/ @backend-team

Frontend code owners

/frontend/ @frontend-team

Security-sensitive code

/auth/ @security-team

CI/CD configuration

/.github/ @devops-team

Purpose: GitHub or GitLab can use code ownership rules to request reviews from the responsible team. Configure the repository's branch protection settings to require code-owner approval.

3. Pull-Request Template

Create .github/pull_request_template.md:

Pull Request Title

Briefly describe the change.

1. Description

- What change is being made?
- Why is it required?

2. Type of Change

- Bug fix
- New feature
- Refactoring
- Documentation

3. Testing

- Automated tests passed
- Manual testing completed
- No new errors introduced

4. Security and Compatibility

- Security implications checked
- Existing functionality verified
- Configuration or database changes documented

5. Screenshots / Evidence

Add screenshots or test results if applicable.

6. Reviewer Checklist

- Code is readable and maintainable
- Error handling is appropriate
- Tests cover the changes
- Documentation is updated where necessary

4. PR Review Workflow

Developer creates feature branch

Open Pull Request

CI Tests + Security Checks

Two Approvals + Code Owner Approval

Merge into main

Example: If a developer changes the login authentication code, the PR requires two approvals, including the designated security code owner's approval when applicable. CI tests and security checks must pass before merging.

Q12. What Is Continuous Integration? Draw and Explain a Typical CI Workflow from Commit to Artifact. List Key CI Practices and Explain Pipeline as Code with Its Advantages. (5 Marks)

Definition: Continuous Integration (CI) is a software development practice in which developers frequently merge code changes into a shared repository. Each change is automatically built and tested to detect errors early.

1. Typical CI Workflow

Developer Writes and Commits Code

Git Repository (GitHub / GitLab)

CI Server (Jenkins / GitHub Actions)

Build and Compile Code

Run Automated Tests

Store Versioned Artifact

Explanation:

1. Commit: A developer pushes code changes to the repository.
2. Trigger: The CI server automatically starts the pipeline.
3. Build: The system compiles the code and checks dependencies.
4. Test: Automated tests detect bugs and errors.
5. Artifact: If the pipeline succeeds, the build output is stored for later deployment.

2. Key CI Practices

1. Maintain a single shared source-code repository.
2. Commit small changes frequently.
3. Automate builds and tests.
4. Keep builds fast and reliable.
5. Fix broken builds immediately.
6. Never ignore failed tests when merging code.

3. Pipeline as Code

Pipeline as Code means defining the CI/CD workflow in a version-controlled file instead of configuring every step manually through a graphical interface.

Examples: .github/workflows/ci.yml for GitHub Actions and Jenkinsfile for Jenkins.

Advantages:

- Consistency: The same process runs repeatedly.
- Version control: Pipeline changes can be reviewed and tracked.
- Automation: Reduces manual work and human errors.
- Reusability: Teams can reuse pipeline definitions.
- Recovery: Earlier versions can be restored when changes cause problems.

Example: When a developer pushes Java code to GitHub, GitHub Actions automatically builds the project, runs unit tests, and stores the resulting JAR file.

Q13. Write a GitHub Actions Workflow for a Java Maven Project. Explain on, jobs, runs-on, steps, uses, and run. (5 Marks)

Definition: GitHub Actions is a CI/CD automation tool that runs workflows in response to events such as code pushes and pull requests.

1. Workflow File: .github/workflows/ci.yml

```
name: Java Maven CI


on:
  push:
    branches: [main]
```

```
  pull_request:
    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up JDK 17 and cache Maven
        uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: '17'
          cache: maven

      - name: Build project
        run: mvn -B -DskipTests package

      - name: Run unit tests
        run: mvn -B test

      - name: Upload test reports
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: junit-test-reports
          path: target/surefire-reports/
          if-no-files-found: warn
```

2. Explanation of Keywords

1. on: Defines the events that trigger the workflow, such as a push or pull request targeting main.

2. jobs: Defines one or more jobs to execute in the workflow.
3. runs-on: Specifies the environment where the job runs, such as ubuntu-latest.
4. steps: Lists the tasks executed in order within a job.
5. uses: Runs a reusable action, such as checking out code or setting up Java.
6. run: Executes a shell command, such as mvn test.

3. Workflow Explanation

- The checkout action downloads the repository code.
- The setup action installs JDK 17 and caches Maven dependencies.
- Maven builds the project and runs unit tests.
- The upload-artifact action saves the test reports, even when earlier steps fail.

Example: When a developer opens a PR targeting main, the workflow builds the Java project, runs unit tests, and uploads available test reports for inspection.

Q14. Write a Declarative Jenkins Pipeline with Checkout, Build, Unit Test, and Publish Stages. Explain Jenkins Keywords and Differentiate Nexus Snapshot and Release Repositories. (5 Marks)

Definition: Jenkins is an automation server used to build, test, and deploy applications. A declarative pipeline defines the complete process in a Jenkinsfile.

1. Jenkinsfile

The following example assumes Maven is configured in Jenkins, Nexus credentials are stored in Maven's configured settings.xml, and pom.xml contains the correct Nexus distributionManagement configuration.

```
pipeline {
  agent any


  environment {
    APP_NAME = 'myapp'
  }


  stages {
    stage('Checkout') {
      steps {
        checkout scm
      }
    }


    stage('Build') {
```

```
        steps {
            sh 'mvn -B clean package -DskipTests'
        }
    }

    stage('Unit Test') {
        steps {
            sh 'mvn -B test'
        }
    }

    stage('Publish') {
        steps {
            sh 'mvn -B deploy -DskipTests'
        }
    }
  }

  post {
    always {
        junit testResults: 'target/surefire-reports/*.xml',
             allowEmptyResults: true
    }

    failure {
        mail to: 'team@example.com',
            subject: "Build Failed: ${env.JOB_NAME}",
            body: "Build ${env.BUILD_NUMBER} failed. Check Jenkins logs."
    }
  }
}
```

2. Explanation of Jenkins Keywords

1. agent: Specifies where the pipeline runs. any allows any available agent.

2. stages: Groups the pipeline into stages such as Build and Unit Test.
3. steps: Contains the commands or actions executed within a stage.
4. environment: Defines environment variables accessible to the pipeline.
5. post: Defines actions that run after the pipeline or a stage completes. Here, test reports are published and failure notifications are sent.

3. Nexus Snapshot vs. Release Repositories

Nexus is a repository manager that stores build artifacts such as JAR files.

Basis	Snapshot Repository	Release Repository
Purpose	Stores development versions	Stores stable, released versions
Version example	1.0-SNAPSHOT	1.0.0
Changes	May be updated repeatedly	Normally immutable after publishing
Usage	Development and testing	Production releases
Stability	Under development	Approved and tested

Example: Developers publish myapp-1.0-SNAPSHOT.jar during development. After testing and approval, they publish myapp-1.0.0.jar to the release repository.

Important: The Maven project must be configured with the appropriate Nexus URLs and credentials. The Jenkins mail step also requires a configured email service and the relevant plugin.

Q15. A Team's CI Pipeline Takes 45 Minutes, Fails Randomly Due to Flaky Tests, Rebuilds the Artifact Separately for Testing and Production, and Developers Merge Without Waiting for the Build. Analyze the Root Causes and Propose at Least Five Improvements. (5 Marks)

Definition: Continuous Integration requires frequent integration of code, reliable automated testing, and fast feedback. A slow or unreliable pipeline reduces developer confidence and increases the risk of defects.

1. Root Causes and Improvements

Problem	Root cause	Proposed improvement
1. Pipeline takes 45 minutes	Slow tests, unnecessary build steps, or inefficient dependency downloads	Parallelize independent tests, cache dependencies, and remove redundant steps.
2. Flaky tests	Tests depend on timing, shared state, external services, or unstable test data	Isolate tests, use controlled test data, and replace unstable dependencies with reliable test doubles where appropriate.
3. Artifact rebuilt for each environment	Testing and production use different build outputs	Build once, version the artifact, and promote the same tested artifact to production.
4. Developers merge before CI finishes	No enforced branch-protection rules or required checks	Require successful CI checks before merging into main.

5. Slow feedback	Developers wait too long to learn whether their changes are safe	Run quick checks first and execute longer tests in parallel or later stages.
6. Repeated pipeline failures	Failures are not analyzed systematically	Track failure causes, assign ownership, and fix recurring issues instead of routinely rerunning them.

2. Recommended CI Workflow

Commit Code and Open PR

Fast Checks: Linting and Compilation

Automated Unit and Integration Tests

Build and Store One Versioned Artifact

Merge After Required Checks Pass

Promote the Same Artifact to Production

3. Expected Benefits

1. Faster feedback for developers.
2. More reliable and repeatable builds.
3. Fewer bugs reaching production.
4. Consistent artifacts across test and production environments.
5. Safer merges and more dependable releases.

Example: A Java team uses caching and parallel tests to shorten its pipeline, fixes flaky tests, and configures GitHub branch protection. Once the build succeeds, the same versioned JAR is tested and then promoted to production without rebuilding it.

Q16. Define Continuous Integration, Continuous Delivery, and Continuous Deployment. Draw a Deployment Pipeline Showing the Manual Approval Gate. (5 Marks)

Definition: CI/CD is a set of software practices that automates code integration, testing, and software delivery to make releases faster and more reliable.

1. Definitions

1. Continuous Integration (CI): Developers frequently merge code into a shared repository, where automated builds and tests check for errors.
2. Continuous Delivery (CD): Code is automatically built and tested and kept ready for release, but production deployment usually requires manual approval.
3. Continuous Deployment: Every change that passes all required automated checks is deployed to production automatically, without a routine manual approval gate.

2. Continuous Delivery Pipeline

Code Commit → Build and Automated Tests → Package and Store Artifact → Manual Approval Gate (Authorized person approves release) → Production Deployment

3. Continuous Deployment Pipeline

Code Commit → Build and Automated Tests → Package and Automated Validation → Automatic Production Deployment

4. Main Difference

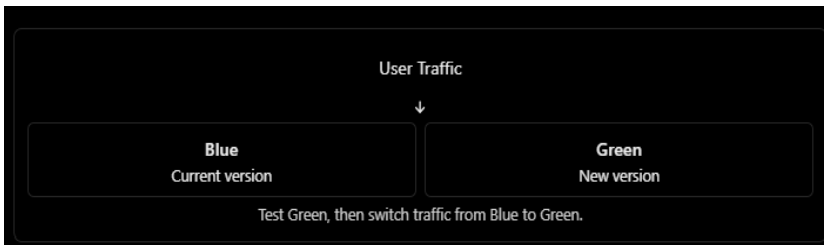
Basis	Continuous Delivery	Continuous Deployment
Production release	Usually requires manual approval	Automatically deployed after checks pass
Automation	Automated build and testing	Automated build, testing, and production deployment
Control	Human decides when to release	Pipeline decides based on predefined rules
Best suited for	Organizations requiring release approval	Teams with strong automation and monitoring

Q17. Explain Blue/Green Deployment and Canary Release with Diagrams. Compare Them with Rolling Deployment. Which Is Harder When a Release Includes a Database Schema Change, and Why? (5 Marks)

Definition: Deployment strategies control how a new software version is introduced into production while reducing downtime and the risk of failure.

1. Blue/Green Deployment

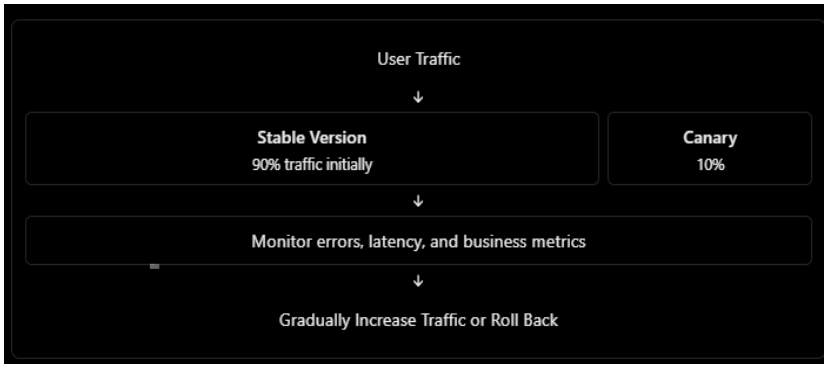
Two production environments are maintained: Blue runs the current version, while Green runs the new version. Traffic is switched to Green after testing.



Example: If the new version fails, traffic can be switched back to Blue, provided the old version remains compatible with the current data.

2. Canary Release

The new version is first released to a small percentage of users. If monitoring shows good results, traffic is gradually increased.



3. Comparison of Deployment Strategies

Criterion	Blue/Green	Canary	Rolling
Infrastructure cost	High; two environments may be needed	Medium; additional capacity for the canary	Usually lower; instances are replaced gradually
Blast radius	Potentially high if all traffic switches to a faulty version	Low initially because only a small group is exposed	Medium; exposure increases as instances are updated
Rollback speed	Very fast traffic switch	Fast if traffic routing is automated	Depends on how quickly old instances can be restored
Monitoring need	Important after traffic switch	Very high; compare new and stable versions	Important throughout the rollout

4. Database Schema Changes

Database changes are difficult with all three strategies, especially when the new schema is incompatible with the old application version.

- Blue/Green: Both versions may share the same database. A destructive schema change can break the old version when traffic switches back.
- Canary: Old and new application versions run simultaneously, so both must work with the database schema.
- Rolling: Old and new application instances coexist during deployment, requiring backward-compatible schema changes.

Recommended practice: Use the expand-and-contract method. First add backward-compatible database fields or tables, deploy code that supports both schemas, migrate the data, and remove old fields only after the old version is no longer needed.

Q18. Evaluate Blue/Green, Canary, Rolling, and Feature-Flag-Based Releases for an E-Commerce Checkout Service Before the Diwali Sale. Recommend a Release and Rollback Strategy. (5 Marks)

Definition: A safe release strategy minimizes downtime, limits customer impact, and allows the team to recover quickly if a new version causes problems.

The company faces a high-risk situation: checkout traffic may reach ten times normal levels, and downtime directly affects revenue.

1. Evaluate the Four Strategies

Strategy	Advantages	Risks / limitations
Blue/Green	Very fast traffic switching and rollback	Requires extra capacity; shared database changes may complicate rollback
Canary	Limits initial exposure and validates the new version under real traffic	Requires strong monitoring and reliable traffic routing

Rolling	Gradually replaces old instances with new ones	Old and new instances coexist; rollback may take longer
Feature flags	New checkout functionality can be enabled or disabled instantly without redeploying	Does not reverse database changes or fix infrastructure failures by itself

2. Recommended Strategy: Canary + Feature Flags

Use a canary release for controlled production testing and feature flags for instant control of the new checkout functionality.

Step 1: Prepare before Diwali

- Load-test the service at ten times normal traffic.
- Test payment failures, retries, timeouts, and duplicate-payment prevention.
- Ensure sufficient server capacity and payment-provider capacity.
- Make database changes backward-compatible.
- Verify monitoring, alerts, and rollback procedures.

Step 2: Release gradually

Deploy New Checkout Version → Enable for 1% of Traffic → Check Payment Success, Errors, and Latency → Increase to 5% → 10% → 25% → 50% → 100%

Increase traffic only after each stage meets predefined health and business-metric thresholds.

3. Rollback Strategy

1. Feature-level failure: Immediately disable the new checkout feature flag and route users to the stable checkout.
2. Service-level failure: Shift canary traffic back to the stable version.
3. Database issue: Use a tested, backward-compatible recovery or migration plan; switching traffic alone may not restore data.
4. Monitoring: Alert the team if payment failures, latency, or error rates exceed defined thresholds.

4. Justification

Criterion	**Evaluation**
Risk	Canary limits the number of customers exposed initially.
Cost	Requires some extra capacity, but generally less than maintaining two full environments.
Rollback time	A feature flag or traffic switch can act within seconds when properly configured.
Observability	Metrics reveal whether the new version harms checkout success or performance.

Example: If payment failures rise after enabling the new checkout for 5% of users, the team disables the feature flag, restores the stable user experience, investigates the issue, and retries only after the fix passes testing.

Q19. What Are Feature Flags? Explain Four Types with Examples. Write Pseudocode for Enabling a New Payment Page for 10% of Users with Instant Disabling. Explain Flag Debt and How Flags Support Release Automation and Rollback. (5 Marks)

Definition: Feature flags are configuration switches that allow developers to enable or disable application features without deploying new code.

1. Four Types of Feature Flags

1. Release Flag: Controls whether a new feature is visible.
    - Example: Enable a redesigned checkout page only after it is ready for release.
2. Experiment Flag: Enables A/B testing of different versions.
    - Example: Show two different payment-page designs to compare conversion rates.
3. Operations Flag / Kill Switch: Quickly disables a feature that causes problems.
    - Example: Disable online payments temporarily if a payment provider becomes unstable.
4. Permission Flag: Enables features only for selected users or roles.
    - Example: Allow administrators to access advanced reports while regular users cannot.

2. Pseudocode: Enable Payment Page for 10% of Users

```
paymentFlag = true

function showPaymentPage(user):
  if paymentFlag == false:
    return showOldPaymentPage()
  if stableBucket(user.id) < 10:
    return showNewPaymentPage()
  return showOldPaymentPage()
function disablePaymentPage():
  paymentFlag = false
```

Explanation:

- paymentFlag controls whether the new payment page is enabled.
- stableBucket(user.id) assigns each user a consistent value from 0 to 99.
- Users assigned values from 0 to 9 see the new payment page, giving approximately 10% exposure.
- Setting paymentFlag = false disables the new page for all users without redeploying the application.

In a real system, the flag should be managed through a reliable configuration service, with appropriate access controls and fast propagation.

3. What Is Flag Debt?

Flag debt occurs when old or unnecessary feature flags remain in the codebase after they are no longer needed.

Problems caused by flag debt:

- Makes code more complicated.
- Increases the number of possible test combinations.
- Creates confusion about which features are active.
- Can cause unexpected behavior if old flags conflict.

How to manage it:

1. Assign an owner to every flag.
2. Record its purpose and planned removal date.
3. Monitor which flags are still used.
4. Remove temporary flags after full rollout.
5. Include flag cleanup in regular maintenance.

4. How Feature Flags Support Release Automation and Rollback

- Release automation: CI/CD can deploy code while the feature remains disabled until approved.
- Gradual rollout: Enable a feature for 1%, 10%, and then 100% of users.
- Instant rollback: Disable a faulty feature without rebuilding or redeploying the entire application.
- Safer experiments: Compare feature versions using real-user metrics.

Example: A team deploys a new payment page with its flag disabled. After automated tests pass, it enables the feature for 10% of users. If payment failures increase, the team disables the flag immediately.

Q20. Explain the principles of Infrastructure as Code. Differentiate mutable and immutable infrastructure. Why is environment parity important? (5 Marks)

1. Definition

Infrastructure as Code (IaC) is the practice of managing and creating IT infrastructure using code instead of manually configuring servers, networks, and other resources.

2. Principles of IaC

1. Declarative vs. Imperative: Declarative code describes the desired final state, while imperative code specifies the steps to achieve it.
2. Idempotence: Running the same code multiple times produces the same desired result without unnecessary changes.
3. Version Control: Infrastructure code is stored in Git to track changes, review updates, and restore previous versions.
4. Reproducibility: The same code can create similar infrastructure whenever required.

5. Automation: Infrastructure can be created, updated, and removed automatically, reducing human errors.

3. Mutable vs. Immutable Infrastructure

Mutable Infrastructure	Immutable Infrastructure
Existing servers are modified.	Existing servers are replaced with new ones.
Changes are applied directly.	A new image or server is created with changes.
Configuration drift may occur.	Provides more consistent environments.
Example: Updating Nginx on an existing server.	Example: Creating a new server image with updated Nginx and replacing the old server.

4. Importance of Environment Parity

Environment parity means keeping development, testing/staging, and production environments as similar as possible.

- Reduces the "works on my machine" problem.
- Helps identify bugs before production.
- Makes testing more reliable.
- Reduces deployment failures.

Example: If the same Nginx version and application dependencies are used in development, staging, and production, the application is more likely to behave consistently.

Q21. Write a Terraform configuration to create an AWS EC2 instance with variables, a security group allowing ports 22 and 80, and an output for the public IP. Explain Terraform commands, state file, and remote backend. (5 Marks)

1. Definition

Terraform is an Infrastructure as Code tool used to create and manage cloud resources through configuration files.

2. Terraform Configuration

Create a file named main.tf:

```
terraform {
 required_providers {
  aws = {
   source  = "hashicorp/aws"
   version = "~> 5.0"
  }
 }
}
```

```
variable "region" {
  default = "ap-south-1"
}

variable "instance_type" {
  default = "t3.micro"
}

variable "ami_id" {
  description = "Valid AMI ID for the selected region"
  type        = string
}

provider "aws" {
  region = var.region
}

resource "aws_security_group" "web_sg" {
  name = "web-security-group"

  ingress {
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
```

```
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

resource "aws_instance" "web" {
  ami                    = var.ami_id
  instance_type          = var.instance_type
  vpc_security_group_ids = [aws_security_group.web_sg.id]

  tags = {
    Name = "WebServer"
  }
}

output "public_ip" {
  value = aws_instance.web.public_ip
}
```

Note: Supply a valid AMI ID for the selected AWS region. For security, SSH port 22 should normally be restricted to your own IP instead of being open to everyone.

3. Terraform Workflow

terraform init — Initializes the project and downloads providers.

terraform plan — Shows the proposed infrastructure changes.

terraform apply — Creates or updates the resources.

terraform destroy — Removes the resources managed by the configuration.

4. State File and Remote Backend

- State file (terraform.tfstate): Records Terraform's knowledge of managed resources and maps them to real infrastructure.

- Remote backend: Stores state in a shared remote location, such as an S3 bucket, so team members can collaborate. State locking helps prevent simultaneous conflicting changes.
- State files may contain sensitive information, so access must be restricted.

Q22. Write an Ansible playbook to install Nginx, deploy an index.html using a Jinja2 template, ensure the service is started and enabled, and restart Nginx only when configuration changes. Explain Ansible concepts. (5 Marks)

1. Definition

Ansible is an automation and configuration management tool used to install software, configure servers, and deploy applications. It is agentless and commonly connects to Linux servers through SSH.

2. Inventory File

Create inventory.ini:

```
[webservers]
web1 ansible_host=192.168.1.10
web2 ansible_host=192.168.1.11
```

3. Jinja2 Template

Create templates/index.html.j2:

```
<!DOCTYPE html>
<html>
<head>
  <title>{{ site_title }}</title>
</head>
<body>
  <h1>{{ welcome_message }}</h1>
</body>
</html>
```

4. Ansible Playbook

Create site.yml:

```
---
- name: Configure web servers
 hosts: webservers
 become: yes
```

```
vars:
  site_title: "My Website"
  welcome_message: "Welcome to Nginx!"

tasks:
  - name: Install Nginx
    ansible.builtin.apt:
      name: nginx
      state: present
      update_cache: yes

  - name: Deploy website
    ansible.builtin.template:
      src: templates/index.html.j2
      dest: /var/www/html/index.html
      mode: '0644'

  - name: Ensure Nginx is started and enabled
    ansible.builtin.service:
      name: nginx
      state: started
      enabled: yes

handlers:
  - name: Restart Nginx
    ansible.builtin.service:
      name: nginx
      state: restarted
```

To restart Nginx only when its configuration changes, add the following notify to the task that manages the Nginx configuration file:

```
    notify: Restart Nginx
```

The handler runs only when its notified task reports a change. The website deployment task above does not notify the handler, because changing index.html alone does not require an Nginx restart.

Run the playbook:

ansible-playbook -i inventory.ini site.yml

5. Important Ansible Concepts

1. Inventory: Lists the servers Ansible manages.
2. Modules: Perform operations such as installing packages and managing services.
3. Tasks: Individual actions performed on target servers.
4. Handlers: Special tasks triggered by notifications, such as restarting Nginx after a configuration change.
5. Agentless: Ansible usually connects over SSH without installing a separate agent on managed Linux servers.
6. Idempotent: Repeated execution maintains the desired state without making unnecessary changes.

Q23. "We already use Ansible, so we don't need Terraform." Evaluate this claim by comparing Terraform and Ansible. Recommend how both should be used together in one pipeline. (5 Marks)

1. Definition

The claim is incorrect because Terraform and Ansible perform different but complementary tasks. Terraform mainly provisions infrastructure, while Ansible configures servers and deploys software.

2. Terraform vs. Ansible

Basis	Terraform	Ansible
Purpose	Creates infrastructure, such as EC2 instances and networks.	Configures servers and installs software.
Style	Declarative: describes the desired infrastructure state.	Uses task-based instructions to reach the desired state, with declarative modules.
State management	Maintains a state file to track managed resources.	Generally does not require a central infrastructure state file.
Drift handling	terraform plan detects differences between configuration, state, and observed resources.	Re-running playbooks can correct configuration differences on managed servers.
Example	Creates an AWS EC2 instance.	Installs Nginx on that EC2 instance.

3. How Both Work Together

A DevOps team can use Terraform and Ansible in the same deployment pipeline.

Step 1: Terraform

Create EC2 instances, networks and security groups. →

Step 2: Ansible

Install Nginx, configure servers and deploy the website. →

Step 3: Testing

Check service status and test website availability. →

Step 4: Deployment

Deploy the application and monitor its performance.

4. Example

For an e-commerce website:

- Terraform creates AWS EC2 instances, networking and security groups.
- Ansible installs Nginx, configures the servers and deploys application files.
- The CI/CD pipeline tests the deployment before releasing it to users.

Recommendation: Use Terraform for infrastructure provisioning and Ansible for configuration management. Together, they improve automation, consistency, and deployment reliability.

Q24. Differentiate containers and virtual machines with an architecture diagram. Explain Docker architecture and image layers/caching. (5 Marks)

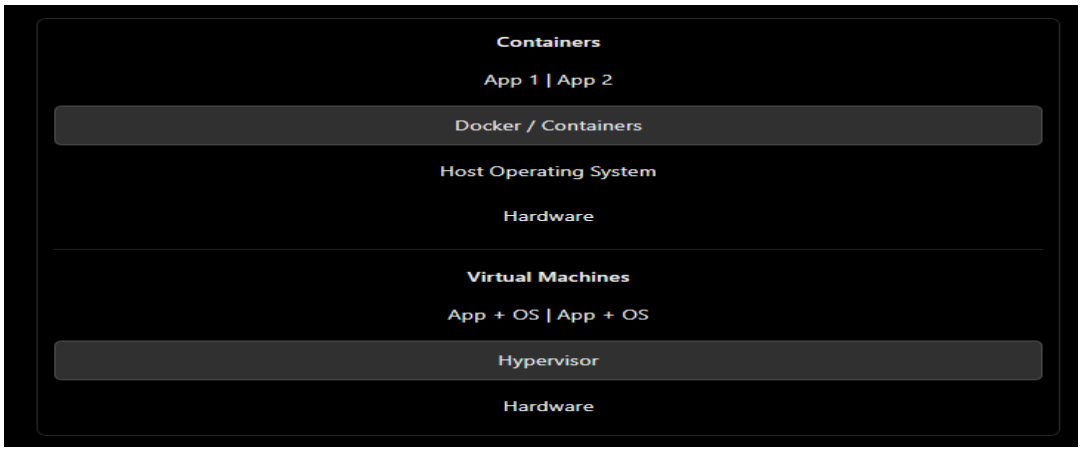
1. Definition

Containers are lightweight, isolated environments that package an application with its dependencies. Virtual Machines (VMs) run complete guest operating systems on virtualized hardware.

2. Containers vs. Virtual Machines

Containers	Virtual Machines
Share the host operating system kernel.	Each VM runs its own guest OS.
Lightweight and start quickly.	Require more resources and start more slowly.
Usually use less memory and storage.	Usually use more memory and storage.
Isolation is provided at the OS level.	Isolation is provided through a hypervisor.
Example: Docker container.	Example: Ubuntu VM running on VMware.

3. Architecture Diagram



4. Docker Architecture

1. Client: Accepts Docker commands.
2. Daemon: Creates and manages containers.
3. Image: A template used to create containers.
4. Container: Runs the application.
5. Registry: Stores images, e.g. Docker Hub.

5. Image Layers and Caching

- Docker images contain multiple layers.
- Docker reuses unchanged layers to save time.
- If a step changes, Docker rebuilds that step and affected later steps.

Example: Copying requirements.txt before application code helps Docker reuse installed dependencies.

Q25. Containerize a Python Flask application that uses PostgreSQL. (a) Write a Dockerfile using a slim base image, dependency caching, and a non-root user. (b) Write a docker-compose.yml with the app and database services, a named volume, environment variables and port mapping. (c) Give the commands to build, tag and push the image to Docker Hub or a private registry.

1. Definition

Docker packages an application and its required libraries into an image so it can run on different computers.

2. (a) Dockerfile

Create a file named Dockerfile.

FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

RUN useradd -m appuser

COPY --chown=appuser:appuser . .

USER appuser

EXPOSE 500

CMD ["gunicorn", "--bind", "0.0.0.0:5000", "app:app"]

Create requirements.txt:

Flask

psycopg[binary]

gunicorn

Explanation:

- FROM selects a small Python image.
- COPY copies the files.
- RUN installs required libraries.
- USER runs the application as a non-root user.
- CMD starts the Flask application.

3. (b) Docker Compose File

Create compose.yaml.

```
services:
  app:
    build: .
    ports:
      - "5000:5000"
    environment:
      DB_HOST: db
      DB_NAME: mydb
      DB_USER: appuser
      DB_PASSWORD: ${DB_PASSWORD:?Set DB_PASSWORD}
    depends_on:
      db:
        condition: service_healthy
  db:
    image: postgres:16
    environment:
      POSTGRES_DB: mydb
      POSTGRES_USER: appuser
      POSTGRES_PASSWORD: ${DB_PASSWORD:?Set DB_PASSWORD}
    volumes:
      - dbdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U appuser -d mydb"]
      interval: 5s
      timeout: 3s
```

```
    retries: 10
volumes:
  dbdata:
```

Store the password in a local .env file and do not upload it to GitHub.

Explanation:

- app runs Flask.
- db runs PostgreSQL.
- ports exposes Flask on port 5000.
- volumes saves database data.
- environment passes database settings to the application.

4. (c) Docker Commands

```
docker compose up -d --build
docker login
docker build -t yourusername/flask-app:1.0 .
docker push yourusername/flask-app:1.0
```

Meaning:

- build creates an image.
- login logs in to Docker Hub.
- push uploads the image.

Q26. Write Kubernetes YAML for a Deployment running 3 replicas of nginx:1.27 with resource limits and a readiness probe, and a Service of type NodePort exposing it. Explain Pod, ReplicaSet, Deployment and the Service types (ClusterIP, NodePort, LoadBalancer). Give kubectl commands to apply, scale to 5 replicas, perform a rolling update and roll back.

1. Definition

Kubernetes is a platform used to run, manage, and scale containers automatically.

2. Kubernetes YAML

Create nginx.yaml.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 3
```

```
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.27
          ports:
            - containerPort: 80
          resources:
            limits:
              cpu: "250m"
              memory: "128Mi"
          readinessProbe:
            httpGet:
              path: /
              port: 80
            initialDelaySeconds: 5
            periodSeconds: 10


---
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  type: NodePort
  selector:
    app: nginx
```

```
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080
```

3. Important Kubernetes Objects

1. Pod: Runs one or more containers.
2. ReplicaSet: Maintains the required number of Pods.
3. Deployment: Creates and updates Pods.
4. Service: Provides network access to Pods.

4. Types of Service

Type	Meaning
ClusterIP	Access inside the cluster.
NodePort	Access through a port on a node.
LoadBalancer	Exposes the application using a load balancer.

5. Important Commands

Apply the file:

kubectl apply -f nginx.yaml

Scale to 5 replicas:

kubectl scale deployment nginx-deployment --replicas=5

Update Nginx:

kubectl set image deployment/nginx-deployment nginx=nginx:1.28

Rollback the update:

kubectl rollout undo deployment/nginx-deployment

Check Pods:

kubectl get pods

Q27. Design the Kubernetes deployment for a three-tier application (React frontend, Spring Boot API, MySQL database). Draw the architecture and specify which Kubernetes objects you use for each tier (Deployment, StatefulSet, Service, Ingress, ConfigMap, Secret, PersistentVolumeClaim, HorizontalPodAutoscaler). Show a Helm chart folder structure for it and explain how values files support dev and prod environments.

1. Definition

A three-tier application has three parts: frontend, backend, and database. Kubernetes manages these parts using different objects.

2. Architecture Diagram

User / Browser → Ingress → React Frontend → Spring Boot API → MySQL Database

3. Kubernetes Objects Used

Object	Purpose
Deployment	Runs React and Spring Boot.
StatefulSet	Manages MySQL.
Service	Connects application components.
Ingress	Allows external web access.
ConfigMap	Stores normal settings.
Secret	Stores passwords and sensitive data.
PVC	Stores database data permanently.
HPA	Automatically increases or decreases replicas.

4. Helm Chart Folder Structure

Helm helps install and manage Kubernetes applications.

```
three-tier-app/
├── Chart.yaml
├── values.yaml
├── values-dev.yaml
├── values-prod.yaml
└── templates/
    ├── frontend.yaml
    ├── backend.yaml
    ├── mysql.yaml
    ├── services.yaml
    ├── ingress.yaml
    └── pvc.yaml
```

5. Values Files

- values.yaml: Contains default settings.
- values-dev.yaml: Contains development settings.
- values-prod.yaml: Contains production settings.

Example values-dev.yaml:

frontend:

replicas: 1

backend:

replicas: 1

Example values-prod.yaml:

frontend:

replicas: 3

backend:

replicas: 3

Install in development:

helm upgrade --install myapp ./three-tier-app -f values-dev.yaml

Install in production:

helm upgrade --install myapp ./three-tier-app -f values-prod.yaml

Benefit: Helm uses the same application configuration templates for different environments while allowing different settings.

Q28. Explain Monitoring vs. Observability and the Three Pillars. Draw Prometheus and Grafana Architecture, write a PromQL query and alert rule, and compare ELK and Loki. (5 Marks)

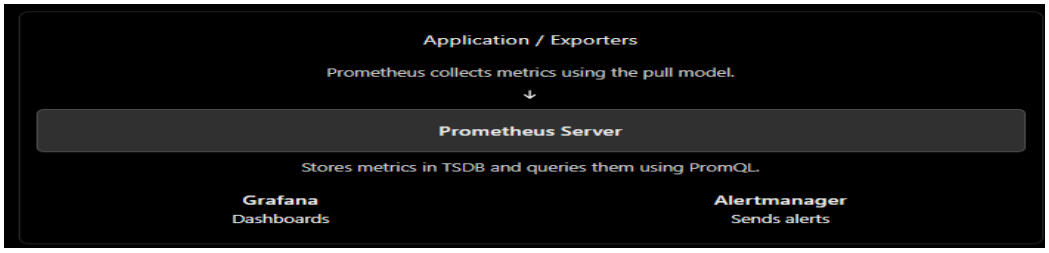
1. Definition

Monitoring means checking the system's health using metrics and alerts. Observability means understanding why a system has a problem by studying its metrics, logs, and traces.

2. Three Pillars of Observability

1. Metrics: Numerical data, such as CPU usage, memory usage, and error rate.
2. Logs: Records of events that happen inside an application.
3. Traces: Show the path of a request through different services.

3. Prometheus and Grafana Architecture



Important components:

- Exporters: Provide metrics from applications and servers.
- Prometheus: Collects and stores metrics.
- TSDB: Stores time-based data.
- PromQL: Queries and calculates metrics.
- Grafana: Displays graphs and dashboards.
- Alertmanager: Sends notifications when alerts fire.

4. PromQL Query for HTTP 5xx Error Rate

Assume the application exposes the http_requests_total counter with a status label.

```
100 *
sum(rate(http_requests_total{status=~"5.."}[5m]))
/
sum(rate(http_requests_total[5m]))
```

Meaning: Calculates the percentage of HTTP 5xx error requests over the last 5 minutes.

5. Alert Rule for More Than 5% Errors

```
groups:
 - name: http-alerts
   rules:
    - alert: HighHTTP5xxErrorRate
      expr: |
       (
         sum(rate(http_requests_total{status=~"5.."}[5m]))
         /
         sum(rate(http_requests_total[5m]))
       ) > 0.05
      for: 5m
      labels:
        severity: critical
      annotations:
        summary: "HTTP 5xx error rate is above 5%"
```

Meaning: The alert fires when the 5xx error rate stays above 5% for 5 minutes. Prometheus evaluates the rule, and Alertmanager handles notifications.

6. ELK vs. Loki

ELK Stack	Loki
Uses Elasticsearch, Logstash and Kibana.	Uses Loki for logs and Grafana for viewing.
Indexes log content.	Mainly indexes log labels.
Can require more storage and resources.	Often more storage-efficient.
Searches detailed log text effectively.	Searches logs using labels and LogQL.

Example: Prometheus detects a high error rate, Grafana displays the problem, and Loki or ELK helps find the error messages causing it.

Q29. A team's data for one month: 40 production deployments; 6 of them caused incidents needing a fix; the 3 production outages were restored in 30, 90 and 60 minutes; lead times for five sampled changes were 2 h, 4 h, 6 h, 24 h and 72 h. Calculate deployment frequency, change failure rate, MTTR and median lead time. Evaluate the team's performance against DORA benchmarks and recommend the single most valuable improvement.

1. Definition

DORA metrics measure a software team's delivery speed, stability, and ability to recover from failures.

2. Calculate the Four Metrics

1. Deployment Frequency

The team made 40 deployments in one month.

$$\text{Frequency} = \frac{40}{1 \text{ month}}$$

Answer: 40 deployments per month.

2. Change Failure Rate (CFR)

6 deployments caused incidents out of 40 deployments.

$$\text{CFR} = \frac{6}{40} \times 100$$

$$= \boxed{15\%}$$

Answer: 15%

3. Mean Time to Restore (MTTR)

The three outages took 30, 90, and 60 minutes to restore.

$$\text{MTTR} = \frac{30 + 90 + 60}{3}$$

$$= \frac{180}{3} = \boxed{60 \text{ minutes}}$$

Answer: 60 minutes (1 hour).

4. Median Lead Time

Lead times in ascending order:

2 h, 4 h, 6 h, 24 h, 72 h

The middle value is 6 hours.

Answer: 6 hours.

3. Final Answers

DORA Metric	Result

Deployment Frequency	40 per month
Change Failure Rate	15%
MTTR	60 minutes
Median Lead Time	6 hours

4. Evaluate Performance

- Deployment Frequency: 40 deployments per month shows regular deployments, but it does not establish an elite level by itself.
- Change Failure Rate: 15% indicates some deployments cause problems; reducing failures should be a priority.
- MTTR: 60 minutes means the team takes an average of one hour to restore service.
- Median Lead Time: 6 hours suggests many changes can reach production quickly, although two sampled changes took much longer.

DORA performance levels depend on the benchmark edition and measurement definitions. These results alone do not prove an exact overall performance level.

5. Most Valuable Improvement

Recommendation: Reduce the Change Failure Rate.

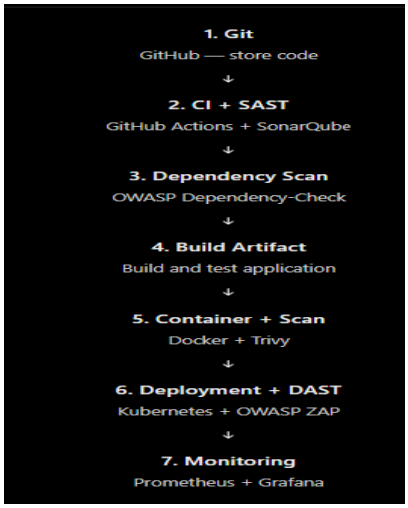
The team should improve automated testing, use smaller changes, and deploy gradually with canary releases and fast rollback. This reduces production incidents and can also reduce restoration time.

Q30. Design an end-to-end DevSecOps pipeline for your mini-project (a web app): Git → CI → artifact → container → deployment → monitoring. Draw the pipeline, name a tool for each stage, and place SAST, dependency scanning, container image scanning and DAST at the right stages with quality gates. Define the KPIs (lead time, deployment frequency, change failure rate, MTTR) you would track and explain how retrospectives on these KPIs drive continuous improvement.

1. Definition

DevSecOps combines Development, Security, and Operations. It adds security checks throughout the software development and deployment process to deliver secure software quickly.

2. DevSecOps Pipeline Diagram



3. Tools and Security Checks

1. Git: GitHub stores source code and tracks changes.
2. CI: GitHub Actions builds the code and runs automated tests.
3. SAST: SonarQube checks source code for security problems before building.
4. Dependency Scanning: OWASP Dependency-Check finds vulnerable libraries.
5. Container Scanning: Trivy checks the Docker image for known vulnerabilities before deployment.
6. DAST: OWASP ZAP tests the running web application for security weaknesses in a test environment.
7. Monitoring: Prometheus collects metrics, and Grafana displays dashboards.

4. Quality Gates

A quality gate is a rule that must pass before the pipeline continues.

- Gate 1: Code review, SAST, and automated tests must pass.
- Gate 2: Dependencies must meet the project's vulnerability policy.
- Gate 3: Container image must pass security scanning.
- Gate 4: DAST must pass before production release.
- Gate 5: Monitor the application after deployment and roll back if serious problems occur.

Critical vulnerabilities or failed tests should stop the pipeline until fixed or formally approved.

5. Important KPIs

KPI	Meaning
Lead Time	Time taken for a code change to reach production.
Deployment Frequency	How often the team deploys to production.
Change Failure Rate	Percentage of deployments that cause failures or require fixes.
MTTR	Average time needed to restore service after a failure.

6. Retrospectives and Continuous Improvement

The team reviews these KPIs regularly to find problems and improve the process.

- High lead time → improve build automation.
- Low deployment frequency → make releases smaller and safer.
- High change failure rate → improve testing and code reviews.
- High MTTR → improve monitoring, alerts, and rollback procedures.

Example: If many deployments fail, the team can add more automated tests and canary releases to reduce production errors.